

**POLITECHNIKA RZESZOWSKA**  
KATEDRA INFORMATYKI I AUTOMATYKI  
DR INŻ. MATEUSZ POMIANEK

**ĆWICZENIE LABORATORYJNE REACT NATIVE 2**

**TypeScript · Komponenty · Nawigacja · REST API**

*Programowanie Aplikacji Mobilnych · moduł RN*

**Projekt: WeatherApp v2 - rozszerzenie projektu z React Native Lab 1**

*Wymagania wstępne: ukończony RN 1 (TypeScript, komponenty, nawigacja, Context API, REST API)*

## Spis treści

|                                                                  |    |
|------------------------------------------------------------------|----|
| 1. Od Context API do Zustand - dlaczego warto przejść? .....     | 1  |
| 2. Zustand - zarządzanie stanem globalnym.....                   | 2  |
| 3. TanStack Query - zarządzanie stanem serwera.....              | 5  |
| 4. AsyncStorage i MMKV - lokalna persystencja danych.....        | 7  |
| 5. Animacje niejawne - Animated API .....                        | 8  |
| 6. Animacje jawne - React Native Reanimated 3 .....              | 10 |
| 7. Testy jednostkowe - Jest i React Native Testing Library ..... | 12 |
| 8. Mockowanie zależności - jest.mock() i MSW.....                | 15 |
| 9. Architektura projektu - Feature-First.....                    | 16 |
| 10. Dostępność i internacjonalizacja .....                       | 18 |
| 11. React Native DevTools - diagnostyka i profilowanie .....     | 20 |

## 1. Od Context API do Zustand - dlaczego warto przejść?

W React Native Lab 1 zarządzaliśmy stanem globalnym przez Context API. Context jest świetny na start, ale przy rozroście aplikacji ujawnia ograniczenia: każda zmiana stanu rerenderuje WSZYSTKICH konsumentów kontekstu, niezależnie od tego, czy zmieniony fragment ma dla nich znaczenie. Zustand rozwiązuje ten problem elegancko i jest jednym z najpopularniejszych rozwiązań stanu globalnego w ekosystemie React w 2024 roku.

**Analogia: biuletyn informacyjny vs. newsletter tematyczny.** Context API to biuletyn firmowy: każdy pracownik dostaje cały newsletter przy każdej zmianie, nawet jeśli interesuje go

tylko kolumna sportowa. Zustand to newsletter tematyczny z subskrypcjami: kazdy subskrybuje tylko te dzialy, ktore go interesuja. Zmiana pogody nie powiadamia czytelnikow sportu.

## 1.1. Problemy Context API w duzych aplikacjach

| Problem z Context API                                                                                                    | Jak Zustand to rozwiazuje                                                                                                                                                 |
|--------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Kazda zmiana dowolnego pola rerenderuje WSZYSTKICH konsumentow kontekstu.                                                | Zustand pozwala subskrybowac sie tylko na wybrane pole: <code>useWeatherStore(s =&gt; s.city)</code> . Zmiana humidity nie rerenderuje komponentu ktory czyta tylko city. |
| Przy wielu kontekstach drzewo Providerow staje sie 'Provider Hell': <code>&lt;A&gt;&lt;B&gt;&lt;C&gt;&lt;D&gt;...</code> | Zustand nie wymaga Providerow. Store to globalny singleton - dostepny z dowolnego miejsca bez owijania aplikacji.                                                         |
| Brak wbudowanego wsparcia dla danych asynchronicznych (loading/error).                                                   | Zustand laczy sie natywnie z TanStack Query do zarzadzania stanem serwera. Kazdy ma swoja role.                                                                           |
| Trudne testowanie - trzeba owijac komponenty w Providery z odpowiednimi wartosciami.                                     | Zustand store mozna zresetowac miedzy testami jednym wywolaniem. Proste i izolowane.                                                                                      |

## 1.2. Konfiguracja Zustand

```
# Instalacja (jeden pakiet, zero konfiguracji):
npm install zustand

# Opcjonalnie - immer dla niemutowalnych aktualizacji (jak w Redux Toolkit):
npm install immer

# Zustand NIE wymaga zadnych zmian w _layout.tsx ani owijania aplikacji!
# Store jest dostepny natychmiast po zaimportowaniu.
```

## 2. Zustand - zarzadzanie stanem globalnym

### 2.1. Tworzenie store - podstawowy wzorzec

| Kod Zustand                                           | Wyjasnienie                                                                    |
|-------------------------------------------------------|--------------------------------------------------------------------------------|
| <code>// stores/favoritesStore.ts</code>              | Jeden plik = jeden store. Konwencja: <code>useFavorites + Store</code> .       |
| <code>import { create } from 'zustand';</code>        | <code>create()</code> to jedyna funkcja z pakietu zustand - tworzy hook store. |
| <code>interface FavoritesState {</code>               | Interfejs opisuje CALY store: stan + akcje razem.                              |
| <code>  favorites: string[];</code>                   | Stan: tablica nazw ulubionych miast.                                           |
| <code>  addCity: (city: string) =&gt; void;</code>    | Akcja: metoda zmieniajaca stan. Typ jak metoda interfejsu.                     |
| <code>  removeCity: (city: string) =&gt; void;</code> | Akcja usuwajaca miasto z listy.                                                |
| <code>  clear: () =&gt; void;</code>                  | Akcja zerujaca caly stan - przydatna w testach.                                |

|                                                                                                                                    |                                                                                                                                                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> }  export const useFavoritesStore = create&lt;FavoritesState&gt;() (set =&gt; ({   favorites: ['Warszawa', 'Krakow'], </pre> | <p><code>create&lt;Typ&gt;()(fn)</code> - podwojne nawiasy: zewnętrzne to middleware (<code>persist</code>), wewnętrzne to fabryka.</p> <p>Stan początkowy - inferencja typów z interfejsu.</p> |
| <pre>   addCity: (city) =&gt; set(state =&gt; ({ </pre>                                                                            | <p><code>set()</code> - JEDYNA droga do aktualizacji stanu. Zawsze tworzy nowy obiekt (niemutowalność).</p>                                                                                     |
| <pre>     favorites: state.favorites.includes(city) </pre>                                                                         | <p>Guard: nie dodawaj duplikatów.</p>                                                                                                                                                           |
| <pre>       ? state.favorites       : [...state.favorites, city]     })), </pre>                                                   | <p>Brak zmiany = Zustand NIE rerenderuje.</p> <p>Spread operator - nowa tablica z nowym miastem.</p>                                                                                            |
| <pre>   removeCity: (city) =&gt; set(state =&gt; ({ </pre>                                                                         | <p>Filtrowanie zamiast mutacji.</p>                                                                                                                                                             |
| <pre>     favorites: state.favorites.filter(c =&gt; c !== city)   })), </pre>                                                      | <p><code>filter()</code> zwraca nową tablicę - Zustand wykryje zmianę.</p>                                                                                                                      |
| <pre>   clear: () =&gt; set({ favorites: [] }), </pre>                                                                             | <p>Skrot: jeśli nie potrzebujesz poprzedniego stanu, podaj obiekt bezpośrednio.</p>                                                                                                             |
| <pre> })); </pre>                                                                                                                  |                                                                                                                                                                                                 |

## 2.2. Selektory - subskrypcja tylko na to co potrzebne

```

// Użycie w komponencie - trzy sposoby, różna wydajność:

// SPOSOB 1: Selektor - NAJLEPSZY (rerender tylko gdy favorites się zmieni)
const favorites = useFavoritesStore(state => state.favorites);
const addCity   = useFavoritesStore(state => state.addCity);

// SPOSOB 2: Destrukturyzacja - ZŁY (rerender przy każdej zmianie store!)
const { favorites, addCity } = useFavoritesStore(); // anti-pattern!

// SPOSOB 3: useShallow - gdy potrzebujesz wielu pól bez rerenderów
import { useShallow } from 'zustand/react/shallow';
const { favorites, addCity, removeCity } = useFavoritesStore(
  useShallow(state => ({ favorites: state.favorites, addCity: state.addCity,
removeCity: state.removeCity })))
);

// Wywołanie akcji (identyczne jak setState w Context):
<Pressable onPress={() => addCity('Gdansk')}>
  <Text>Dodaj do ulubionych</Text>
</Pressable>

```

## 2.3. Middleware persist - automatyczna persystencja

Zustand ma wbudowany middleware persist, który automatycznie zapisuje i wczytuje stan z AsyncStorage. Odpowiednik ręcznie obsługiwanego Hive.box w Dart/Flutter, ale bez generowania kodu.

```
import { create } from 'zustand';
import { persist, createJSONStorage } from 'zustand/middleware';
import AsyncStorage from '@react-native-async-storage/async-storage';

export const useFavoritesStore = create<FavoritesState>() ( // 1. generyk stanu
  persist( // 2. middleware persist
    (set) => ({ // 3. fabryka stanu
      favorites: [],
      addCity: (city) => set(s => ({ favorites: s.favorites.includes(city) ?
s.favorites : [...s.favorites, city] })),
      removeCity: (city) => set(s => ({ favorites: s.favorites.filter(c => c !==
city) })),
      clear: () => set({ favorites: [] }),
    }),
    {
      name: 'favorites-storage', // klucz w AsyncStorage
      storage: createJSONStorage(() => AsyncStorage), // adapter dla RN
      partialize: (state) => ({ favorites: state.favorites }), // zapisz TYLKO dane,
nie akcje
    }
  )
);

// To wszystko! Zustand automatycznie:
// - Odczytuje stan z AsyncStorage przy uruchomieniu aplikacji
// - Zapisuje do AsyncStorage przy każdej zmianie favorites
// - Brak useEffect, brak ręcznie pisanego loadData()/saveData()
```

**Optymalizacja:** persist z MMKV: AsyncStorage jest asynchroniczne i stosunkowo wolne. Dla lepszej wydajności użyj MMKV (react-native-mmkv) - synchroniczne, 10x szybsze od AsyncStorage. Wymiana to zmiana jednej linii: storage: createJSONStorage(() => new MMKV()). MMKV omówione w sekcji 4.

## 2.4. Wzorzec: store asynchroniczny ze stanami loading/error

```
// stores/weatherStore.ts - store z danymi asynchronicznymi
import { create } from 'zustand';
import { fetchWeather } from '../services/weatherService';
import { WeatherData } from '../types/weather';

interface WeatherState {
  // Stan danych
  weatherMap: Record<string, WeatherData>; // miasto -> dane pogodowe
  // Stan ładowania
  loadingCities: Set<string>; // które miasta są aktualnie ładowane
  errors: Record<string, string>; // miasto -> komunikat błędu
  // Akcje
  loadWeather: (city: string) => Promise<void>;
  clearError: (city: string) => void;
}

export const useWeatherStore = create<WeatherState>() ((set, get) => ({
```

```
weatherMap: {},
loadingCities: new Set(),
errors: {},

loadWeather: async (city) => {
  // Guard: nie laduj jesli juz w trakcie ladowania
  if (get().loadingCities.has(city)) return;

  set(s => ({ // Ustaw loading = true
    loadingCities: new Set([...s.loadingCities, city]),
    errors: { ...s.errors, [city]: undefined }, // Skasuj stary blad
  }));

  try {
    const data = await fetchWeather(city);
    set(s => ({
      weatherMap: { ...s.weatherMap, [city]: data },
      loadingCities: new Set([...s.loadingCities].filter(c => c !== city)),
    }));
  } catch (e) {
    set(s => ({
      errors: { ...s.errors, [city]: String(e) },
      loadingCities: new Set([...s.loadingCities].filter(c => c !== city)),
    }));
  }
},

clearError: (city) => set(s => ({ errors: { ...s.errors, [city]: undefined } })),
));

// Selektory pomocnicze (poza hookiem - bez rerenderow przy odczycie):
export const selectWeather = (city: string) => (s: WeatherState) =>
s.weatherMap[city];
export const selectLoading = (city: string) => (s: WeatherState) =>
s.loadingCities.has(city);
export const selectError = (city: string) => (s: WeatherState) => s.errors[city];

// Uzycie w komponencie:
const weather = useWeatherStore(selectWeather('Warszawa')); // WeatherData |
undefined
const loading = useWeatherStore(selectLoading('Warszawa')); // boolean
```

### 3. TanStack Query - zarządzanie stanem serwera

TanStack Query (dawniej React Query) to biblioteka specjalizująca się w zarządzaniu 'stanem serwera' - danymi pobranymi z API. Rozróżnia między stanem klienta (ulubione miasta - Zustand) a stanem serwera (aktualna pogoda - TanStack Query). To rozróżnienie jest kluczowe w profesjonalnych aplikacjach mobilnych.

**Analogia: kelner vs. szef kuchni.** Zustand to kelner: zarządza tym, co jest na sali (stan klienta: koszyk, preferencje, UI). TanStack Query to szef kuchni: wie kiedy danie jest gotowe, jak długo wytrzyma w ciepłe i kiedy trzeba ugotować od nowa (cache, stale time, refetch). Kelner nie pyta szefa kuchni co minute czy jedzenie jest gotowe - dostaje powiadomienie.

#### 3.1. Konfiguracja i podstawowy useQuery

```
# Instalacja:
```

```

npm install @tanstack/react-query

// app/_layout.tsx - konfiguracja QueryClient
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 1000 * 60 * 15, // dane 'świeże' przez 15 minut
      gcTime: 1000 * 60 * 30, // cache trzymany 30 minut po ostatnim użyciu
      retry: 2, // ponow 2 razy przy błędzie sieci
    },
  },
});

export default function RootLayout() {
  return (
    <QueryClientProvider client={queryClient}>
      <FavoritesProvider>
        <Stack>{/* ... */}</Stack>
      </FavoritesProvider>
    </QueryClientProvider>
  );
}

// hooks/useWeatherQuery.ts - zapytanie do API
import { useQuery } from '@tanstack/react-query';
import { fetchWeather } from '../services/weatherService';

export function useWeatherQuery(city: string) {
  return useQuery({
    queryKey: ['weather', city], // Unikalny klucz cache. MUSI zawierać parametry!
    queryFn: () => fetchWeather(city), // Funkcja pobierająca dane
    enabled: city.length > 0, // Nie pobieraj gdy city jest puste
    staleTime: 1000 * 60 * 10, // Dane świeże przez 10 minut (nadpisuje default)
  });
}

// Użycie w komponencie - wszystkie stany wbudowane:
const { data, isLoading, isError, error, refetch } = useWeatherQuery('Warszawa');
//      ^^^^^ ^^^^^^^^^ ^^^^^^^ ^^^^^ ^^^^^^
//      Dane  Ładowanie Błęd  Obiekt  Recznie odśwież

```

### 3.2. Kiedy Zustand, kiedy TanStack Query?

| Typ stanu                                  | Narzędzie                    | Przykład w WeatherApp                                                |
|--------------------------------------------|------------------------------|----------------------------------------------------------------------|
| Stan klienta - trwały, niezależny od sieci | Zustand (+ persist)          | Lista ulubionych miast, motyw (jasny/ciemny), preferencje języka     |
| Stan serwera - pobierany z API, ma TTL     | TanStack Query               | Aktualna pogoda, prognoza 7 dni, lista miast z geolokalizacji        |
| Stan UI - lokalny, efemeryczny             | useState w komponencie       | Rozwinięty/zwinięty akordeon, tekst w polu wyszukiwania, wybrany tab |
| Stan formularza                            | React Hook Form lub useState | Formularz dodawania miasta z walidacją                               |

## 4. AsyncStorage i MMKV - lokalna persystencja danych

Każda aplikacja mobilna potrzebuje przechowywać dane lokalnie: ustawienia, dane użytkownika, cache. React Native oferuje kilka rozwiązań, od prostych klucz-wartość po relacyjne bazy danych.

| Rozwiązanie          | Typ                       | Kiedy używać                                                                 | Wady i zalety                                         |
|----------------------|---------------------------|------------------------------------------------------------------------------|-------------------------------------------------------|
| AsyncStorage         | Klucz-wartość, async      | Proste dane użytkownika, ustawienia, cache tokenów (< 6 MB).                 | Wolne (async I/O). Nie szyfrować. Wbudowany w Expo.   |
| MMKV                 | Klucz-wartość, sync       | Wszystko co AsyncStorage, ale szybciej. Zalecany w nowych projektach.        | 10x szybszy. Synchroniczny API. Wymaga native build.  |
| Expo SecureStore     | Klucz-wartość, szyfrowany | Tokeny JWT, hasła, dane wrażliwe. Używa Keychain (iOS) / Keystore (Android). | Bezpieczny. Limit 2048 bajtów na wartość.             |
| SQLite (expo-sqlite) | Relacyjna baza            | Złożone zapytania, relacje, duże dane strukturalne, offline sync.            | Pełny SQL. Bardziej boilerplate. API async od SDK 50. |
| WatermelonDB         | Relacyjna + reactive      | Duże zestawy danych z synchronizacją z backendem. Lazy loading.              | Leniwa ewaluacja. Skomplikowana konfiguracja.         |

### 4.1. AsyncStorage - bezpośrednie użycie

```
import AsyncStorage from '@react-native-async-storage/async-storage';

// ZAPIS - zawsze string. Obiekty serializuj przez JSON:
await AsyncStorage.setItem('theme', 'dark');
await AsyncStorage.setItem('user', JSON.stringify({ name: 'Jan', age: 25 }));

// ODCZYT - zwraca string | null:
const theme = await AsyncStorage.getItem('theme'); // 'dark' lub null
const raw = await AsyncStorage.getItem('user');
const user = raw ? JSON.parse(raw) : null; // deserializacja

// USUWANIE:
await AsyncStorage.removeItem('theme');

// GRUPOWE operacje - batch (szybszy niż wiele pojedynczych):
await AsyncStorage.multiSet([['key1', 'val1'], ['key2', 'val2']]);
const values = await AsyncStorage.multiGet(['key1', 'key2']);
// values = [['key1', 'val1'], ['key2', 'val2']]

// WSZYSTKIE klucze:
const keys = await AsyncStorage.getAllKeys();

// CZYSZCZENIE (uwaga: usuwa wszystkie dane aplikacji!):
await AsyncStorage.clear();
```

### 4.2. MMKV - szybsza alternatywa

MMKV (Memory Mapped Key-Value) to biblioteka od Tencent, oryginalnie napisana dla WeChat. Jest synchroniczna (brak async/await), co upraszcza kod i eliminuje race conditions.

```
# Instalacja MMKV:
npm install react-native-mmkv
npx expo prebuild # Wymagany native build (nie dziala w Expo Go!)

// Tworzenie instancji (singleton):
import { MMKV } from 'react-native-mmkv';
export const storage = new MMKV({ id: 'weather-app-storage' }); // id = izolacja

// SYNCHRONICZNE API - brak await!
storage.set('theme', 'dark'); // zapis string
storage.set('fontSize', 16); // zapis number
storage.set('notifications', true); // zapis boolean
storage.set('user', JSON.stringify({})); // obiekty: JSON

const theme = storage.getString('theme'); // string | undefined
const size = storage.getNumber('fontSize'); // number | undefined
const notif = storage.getBoolean('notifications'); // boolean | undefined

storage.delete('theme'); // usuwanie
storage.clearAll(); // czyszczenie

// Integracja z Zustand persist (zamiana AsyncStorage na MMKV):
import { StateStorage } from 'zustand/middleware';

const mmkvStorage: StateStorage = {
  getItem: (name) => storage.getString(name) ?? null,
  setItem: (name, value) => storage.set(name, value),
  removeItem: (name) => storage.delete(name),
};

// W persist middleware:
storage: createJSONStorage(() => mmkvStorage),
```

## 5. Animacje niejawne - Animated API

React Native ma wbudowane Animated API do tworzenia animacji. Dziala podobnie do AnimatedController z Fluttera - tworzysz wartosci animowane (Animated.Value) i przyłączasz je do właściwości komponentów. Animacje wykonywane są natywnie (poza wątkiem JS) co gwarantuje 60fps nawet przy obciążonej logice.

**Animacje niejawne vs jawne:** W React Native rozróżniamy Animated API (odpowiednik AnimatedContainer/AnimatedOpacity z Fluttera - prostsze) oraz Reanimated 3 (odpowiednik AnimationController - bardziej zaawansowane, pełna kontrola, worklet na wątku UI). Zaczynij od Animated API dla prostych animacji, przejdź do Reanimated gdy potrzebujesz gestów lub skomplikowanych sekwencji.

### 5.1. Animated.Value i podstawowe animacje

| Kod React Native                                                                              | Wyjasnienie                                                                                                                                                                      |
|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>const opacity = useRef(new Animated.Value(0)).current;  Animated.timing(opacity, {</pre> | <p>useRef zapobiega tworzeniu nowej wartości przy każdym renderze. .current dostaje sama wartość.</p> <p>timing() = liniowa animacja do wartości docelowej w zadanym czasie.</p> |



|                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> toValue: 1, duration: 400, easing: Easing.inOut(Easing.ease), useNativeDriver: true,  }).start(); </pre> | <p>Wartosc docelowa animacji (od 0 do 1 = pelna nieprzezroczystosc).</p> <p>Czas trwania w milisekundach. 200-400ms to 'naturalny' zakres.</p> <p>Krzywa animacji. Odpowiednik Curves.easeInOut z Fluttera.</p> <p>KLUCZOWE: true = animacja na watku UI (60fps). false = na watku JS (laguje). Zawsze true gdy tylko mozliwe.</p> <p>.start() uruchamia animacje. Opcjonalnie callback: .start(() =&gt; console.log('gotowe')).</p> |
| <pre> &lt;Animated.View style={{ opacity }}&gt; </pre>                                                         | <p>Animated.View = natywny widok z animowanymi wlasciwosciami. Tylko Animated.* komponenty przyjmuja Animated.Value.</p>                                                                                                                                                                                                                                                                                                             |

## 5.2. Animacje wejścia listy z staggered delay

```

// Animowane wejście kart - każda karta przesuwa się z prawa z opóźnieniem
const CardItem: React.FC<{ weather: WeatherData; index: number }> = ({ weather, index
}) => {
  const translateX = useRef(new Animated.Value(80)).current; // startowe przesunięcie
  const opacity    = useRef(new Animated.Value(0)).current;

  useEffect(() => {
    // Staggered delay: karta N czeka index * 80ms
    const delay = index * 80;

    Animated.parallel([ // parallel = animuj jednocześnie translateX i
opacity
      Animated.timing(translateX, {
        toValue: 0, // docelowo: brak przesunięcia
        delay,
        duration: 350,
        easing: Easing.out(Easing.back(1.5)), // efekt lekkiego 'odbicia' na końcu
        useNativeDriver: true,
      }),
      Animated.timing(opacity, {
        toValue: 1,
        delay,
        duration: 300,
        useNativeDriver: true,
      }),
    ]).start();
  }, []);

  return (
    <Animated.View style={{ opacity, transform: [{ translateX }] }}>
      <WeatherCard weather={weather} />
    </Animated.View>
  );
};

```

## 5.3. Animated.spring - efekt sprężyny

```

// Scale przy naciśnięciu - efekt 'kliknięcia'
const PressableCard: React.FC<WeatherCardProps> = ({ weather, onPress }) => {
  const scale = useRef(new Animated.Value(1)).current;

  const handlePressIn = () => {

```

```
Animated.spring(scale, {
  toValue: 0.95,          // zmniejsz do 95%
  useNativeDriver: true,
  speed: 50,              // szybkość sprężynowania
  bounciness: 0,          // brak odbicia (0 = jak easeOut)
}).start();
};

const handlePressOut = () => {
  Animated.spring(scale, {
    toValue: 1,            // wróć do 100%
    useNativeDriver: true,
    speed: 50,
    bounciness: 4,          // lekkie odbicie przy powrocie
  }).start();
};

return (
  <Pressable onPress={onPress} onPressIn={handlePressIn}
onPressOut={handlePressOut}>
    <Animated.View style={[styles.card, { transform: [{ scale }] }]}>
      <WeatherCard weather={weather} />
    </Animated.View>
  </Pressable>
);
};
```

## 6. Animacje jawne - React Native Reanimated 3

React Native Reanimated 3 to zaawansowana biblioteka animacji działająca w całości na wątku UI (Worklets). Umożliwia płynne animacje gestów bez komunikacji z wątkiem JS - odpowiednik AnimationController + GestureDetector z Fluttera, ale znacznie bardziej ergonomiczny.

**Analogia: pilot zdalnego sterowania vs. autopilot.** Animated API to pilot zdalnego sterowania: twój kod JS wysyła komendy przez radio (bridge) do wątku UI. Gdy JS jest zajęty (parsowanie JSON, renderowanie), komendy się opóźniają - animacja laguje. Reanimated Worklets to autopilot: kod animacji działa bezpośrednio w samolocie (wątku UI) bez oczekiwania na sygnał z zewnątrz. 60fps gwarantowane nawet gdy JS jest zajęty.

### 6.1. Konfiguracja i podstawy Reanimated

```
# Instalacja:
npm install react-native-reanimated

# Konfiguracja Babel (babel.config.js) - Reanimated MUSI być ostatnim pluginem!
module.exports = {
  presets: ['babel-preset-expo'],
  plugins: ['react-native-reanimated/plugin'], // MUSI być na koncu!
};

# Restart Metro z wyczyszczeniem cache po konfiguracji Babel:
npx expo start --clear
```

### 6.2. useSharedValue i useAnimatedStyle - podstawowy wzorzec

| Reanimated 3                                                                                                                                           | Wyjasnienie                                                                                                                                                                                                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>import Animated, {   useSharedValue,   useAnimatedStyle,   withTiming, withSpring,   withDelay,   Easing, } from 'react-native-reanimated';</pre> | <p>Reanimated eksportuje zastepczy Animated - zamien import z react-native.</p> <p>useSharedValue = odpowiednik Animated.Value, ale zyje na watku UI.</p> <p>Funkcje animacji - odpowiedniki Animated.timing() itp.</p>            |
| <pre>const offset = useSharedValue(0);</pre>                                                                                                           | <p>SharedValue = wartosc wspoldzielona miedzy watkiem JS i UI. Brak rerenderow przy zmianie.</p>                                                                                                                                   |
| <pre>const style = useAnimatedStyle(() =&gt; ({   transform: [{ translateX: offset.value }], }));</pre>                                                | <p>useAnimatedStyle = HOF zwracajacy styl na podstawie SharedValues. Wykonuje sie na watku UI.</p> <p>.value = odczyt wartosci. Zawsze uzyj .value w workletach.</p> <p>Wynik jest uzywany jako style={style} w Animated.View.</p> |
| <pre>// Uruchomienie animacji: offset.value = withTiming(100, { duration: 400 });</pre>                                                                | <p>Przypisanie withTiming/withSpring do .value uruchamia animacje.</p> <p>Animacja do 100 w 400ms. Brak .start() - automatyczne.</p>                                                                                               |
| <pre>offset.value = withSpring(-100, { damping: 12 });</pre>                                                                                           | <p>Sprezynowa animacja do -100. damping = tlumienie sprezynowania.</p>                                                                                                                                                             |

### 6.3. Obsluga gestow - Gesture Handler + Reanimated

```
# Instalacja Gesture Handler (wymagana wspolpraca z Reanimated):
npm install react-native-gesture-handler

// Owin aplikacje w GestureHandlerRootView (app/_layout.tsx):
import { GestureHandlerRootView } from 'react-native-gesture-handler';
<GestureHandlerRootView style={{ flex: 1 }}>
  <Stack />{/* reszta aplikacji */}
</GestureHandlerRootView>

// Swipeable card - ciagniecie gestem w lewo (usuwanie z ulubionych):
import { Gesture, GestureDetector } from 'react-native-gesture-handler';
import Animated, { useSharedValue, useAnimatedStyle,
  withTiming, withSpring, runOnJS } from 'react-native-reanimated';

const SwipeableCard: React.FC<{ city: string; onDelete: () => void }> = ({ city,
onDelete }) => {
  const translateX = useSharedValue(0);
  const SWIPE_THRESHOLD = -100; // Po przesunieciu o 100px w lewo - usun

  const panGesture = Gesture.Pan()
    .activeOffsetX([-10, 10]) // Aktywuj gest po 10px przesuniecia (rozroznia od
scroll)
    .onUpdate((event) => {
```

```

    translateX.value = Math.min(0, event.translationX); // Tylko w lewo (min 0)
  })
  .onEnd((event) => {
    if (event.translationX < SWIPE_THRESHOLD) {
      // Przeleciec poza ekran i wywolac usuwanie
      translateX.value = withTiming(-500, { duration: 200 });
      runOnJS(onDelete)(); // Wywolaj JS callback z watku UI
    } else {
      translateX.value = withSpring(0); // Wróc na miejsce
    }
  });

const animatedStyle = useAnimatedStyle(() => ({
  transform: [{ translateX: translateX.value }],
}));

return (
  <GestureDetector gesture={panGesture}>
    <Animated.View style={[styles.card, animatedStyle]}>
      <Text>{city}</Text>
    </Animated.View>
  </GestureDetector>
);
};

```

**runOnJS():** runOnJS() - kluczowa funkcja: Worklets działają na watku UI bez dostępu do stanu JS (useState, Zustand). Aby wywołać funkcję JS z Workletu (np. usunąć element ze store), użyj runOnJS(mojCallback)(). Jest to 'most' z watku UI do watku JS.

## 7. Testy jednostkowe - Jest i React Native Testing Library

React Native używa Jest jako test runnera i React Native Testing Library (RNTL) do testowania komponentów. W odróżnieniu od testów instrumentowanych Androida (emulator wymagany), testy RNTL działają na Node.js - są szybkie i nie wymagają urządzenia.

**Jak RNTL testuje bez emulatora?** React Native Testing Library renderuje komponenty w środowisku testowym (jest-native) które symuluje drzewo komponentów bez prawdziwego silnika renderującego. Sprawdza drzewo semantyczne, teksty i interakcje przez akcesory. Szybkie porównanie: testy jednostkowe Jest ~1ms, testy RNTL ~50-200ms, testy instrumentowane Detox ~5-30s na emulatorze.

### 7.1. Konfiguracja środowiska testowego

```

# Instalacja (jesli nie ma z Expo):
npm install --save-dev @testing-library/react-native
npm install --save-dev @testing-library/jest-native # dodatkowe matchery

# jest.config.js (lub 'jest' w package.json):
module.exports = {
  preset: 'jest-expo', // preset Expo z konfiguracją React Native
  setupFilesAfterFramework: ['@testing-library/jest-native/extend-expect'],
  transformIgnorePatterns: [
    'node_modules/(?!(jest-)?react-native|@react-native(-community)?|expo|expo|zustand)'
  ],
};

```

## 7.2. Anatomia testu komponentu

```
// __tests__/WeatherCard.test.tsx
import React from 'react';
import { render, screen, fireEvent } from '@testing-library/react-native';
import { WeatherCard } from '../components/WeatherCard';
import { WeatherData } from '../types/weather';

// Dane testowe (fixture) - poza describe, wspoldzielone miedzy testami
const mockWeather: WeatherData = {
  city: 'Warszawa',
  temperature: 22.5,
  humidity: 65,
  windSpeed: 12.0,
  weatherCode: 1,
  updatedAt: new Date('2024-01-15'),
};

describe('WeatherCard', () => {

  it('wyswietla nazwe miasta', () => {
    render(<WeatherCard weather={mockWeather} />);

    // screen.getByText - znajdz element po tekście (rzuca blad jesli nie znalazl)
    expect(screen.getByText('Warszawa')).toBeTruthy();
  });

  it('wyswietla temperature z jednostka', () => {
    render(<WeatherCard weather={mockWeather} />);
    expect(screen.getByText('22.5°C')).toBeTruthy();
  });

  it('wywoluje onPress po tapnięciu', () => {
    const mockOnPress = jest.fn(); // funkcja-szpieg

    render(<WeatherCard weather={mockWeather} onPress={mockOnPress} />);

    // fireEvent.press = symulacja tapnięcia
    fireEvent.press(screen.getByText('Warszawa'));

    expect(mockOnPress).toHaveBeenCalledTimes(1);
  });

  it('nie wyswietla przycisku onPress gdy prop nie podany', () => {
    render(<WeatherCard weather={mockWeather} />);
    // queryById zwraca null zamiast rzucac blad - bezpieczne dla 'nie istnieje'
    expect(screen.queryById('press-action')).toBeNull();
  });
});
```

## 7.3. Finders, Queries i Events - API RNTL

| Query    | Zachowanie gdy nie znaleziono | Kiedy uzywac                                   |
|----------|-------------------------------|------------------------------------------------|
| getBy*() | Rzuca blad (fail testu)       | Gdy element MUSI istniec. Najczesciej uzywane. |

|               |                                 |                                                                   |
|---------------|---------------------------------|-------------------------------------------------------------------|
| queryBy*()    | Zwraca null                     | Gdy element może nie istnieć. Używaj z <code>.toBeNull()</code> . |
| findBy*()     | Czeka (async), rzuca po timeout | Elementy pojawiające się asynchronicznie (po fetch).              |
| getAllBy*()   | Rzuca gdy nie ma żadnego        | Gdy oczekujesz wielu elementów.                                   |
| queryAllBy*() | Zwraca []                       | Lista może być pusta - bezpieczne.                                |
| findAllBy*()  | Czeka (async), zwraca []        | Lista asynchroniczna.                                             |

| Selektor            | Co szuka                                        | Priorytet (wg RNTL)                             |
|---------------------|-------------------------------------------------|-------------------------------------------------|
| ByRole()            | Rola dostępności (button, text, textbox)        | 1 - Najwyższy. Testuje tak jak użytkownicy.     |
| ByLabelText()       | Etykieta accessibilityLabel                     | 2 - Testuje dostępność.                         |
| ByText()            | Widoczny tekst (dokładne dopasowanie lub regex) | 3 - Często używany.                             |
| ByPlaceholderText() | Tekst placeholder w TextInput                   | 4 - Dla formularzy.                             |
| ByTestId()          | Prop testID='...' na komponencie                | 5 - Ostatnia deska ratunku. Unikaj jeśli można. |

## 7.4. Testowanie asynchroniczności

```
// __tests__/HomeScreen.test.tsx - testowanie useQuery/useEffect
import { render, screen, waitFor, fireEvent } from '@testing-library/react-native';
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';

// Helper: owiń komponent w wymagane providery
const createWrapper = () => {
  const queryClient = new QueryClient({
    defaultOptions: { queries: { retry: false } }, // bez retry w testach!
  });
  return ({ children }: { children: React.ReactNode }) => (
    <QueryClientProvider client={queryClient}>{children}</QueryClientProvider>
  );
};

it('wyswietla dane po zaladowaniu', async () => {
  render(<HomeScreen />, { wrapper: createWrapper() });

  // Na poczatku: spinner ladowania
  expect(screen.getByRole('progressbar')).toBeTruthy(); // ActivityIndicator

  // waitFor: czekaj az element sie pojawi (max 1000ms domyslnie)
  await waitFor(() => {
    expect(screen.getByText('Warszawa')).toBeTruthy();
  });

  // Po zaladowaniu: spinner zniknal
  expect(screen.queryByRole('progressbar')).toBeNull();
});

it('wyswietla blad przy bledzie sieci', async () => {
  render(<HomeScreen city='NIEISTNIEJACE_MIASTO' />, { wrapper: createWrapper() });
```

```
await waitFor(() => {
  expect(screen.getByText(/Bład/)).toBeTruthy(); // regex: zawiera 'Bład'
});
});
```

## 8. Mockowanie zaleznosci - jest.mock() i MSW

Testy powinny byc szybkie, izolowane i powtarzalne. Prawdziwe wywolania HTTP sa powolne i niedeterministyczne (API moze byc niedostepne). Mockowanie pozwala zastapic zewnetrzne zaleznosci kontrolowanymi obiektami testowymi.

### 8.1. jest.mock() - mockowanie modulow

```
// Mockowanie calego modulu serwisu:
jest.mock('../services/weatherService', () => ({
  fetchWeather: jest.fn(), // Pusta funkcja-szpieg
}));

// Import mockowanego modulu - bedziemy konfigurowac odpowiedzi:
import { fetchWeather } from '../services/weatherService';
const mockFetchWeather = fetchWeather as jest.MockedFunction<typeof fetchWeather>;

// W tekscie - konfiguracja odpowiedzi:
describe('useWeatherQuery', () => {
  beforeEach(() => {
    jest.clearAllMocks(); // Resetuj liczniki i konfiguracje miedzy testami
  });

  it('zwraca dane przy sukcesie', async () => {
    // Arrange: skonfiguruj co mock ma zwrocic
    mockFetchWeather.mockResolvedValue({
      city: 'Warszawa', temperature: 22.5, humidity: 65,
      windSpeed: 12, weatherCode: 1, updatedAt: new Date(),
    });

    // Act: uruchom hook przez renderHook
    const { result } = renderHook(() => useWeatherQuery('Warszawa'),
      { wrapper: createWrapper() }
    );

    // Assert: czekaj na sukces
    await waitFor(() => expect(result.current.isSuccess).toBe(true));
    expect(result.current.data?.city).toBe('Warszawa');
    expect(mockFetchWeather).toHaveBeenCalledTimes(1);
  });

  it('zwraca blad przy niepowodzeniu sieci', async () => {
    mockFetchWeather.mockRejectedValue(new Error('Network Error'));

    const { result } = renderHook(() => useWeatherQuery('Warszawa'),
      { wrapper: createWrapper() }
    );

    await waitFor(() => expect(result.current.isError).toBe(true));
    expect(result.current.error?.message).toBe('Network Error');
  });
});
```

## 8.2. Testowanie Zustand store

```
// __tests__/favoritesStore.test.ts
import { useFavoritesStore } from '../stores/favoritesStore';
import { act, renderHook } from '@testing-library/react-native';

// Resetuj store miedzy testami (Zustand persystuje miedzy testami w tym samym
// pliku!)
beforeEach(() => {
  useFavoritesStore.setState({ favorites: [] }); // Czyszcimy stan
});

it('dodaje miasto do ulubionych', () => {
  const { result } = renderHook(() => useFavoritesStore());

  act(() => {
    result.current.addCity('Gdansk'); // act() = 'otocz akcje zmieniajaca stan'
  });

  expect(result.current.favorites).toContain('Gdansk');
  expect(result.current.favorites).toHaveLength(1);
});

it('nie dodaje duplikatow', () => {
  const { result } = renderHook(() => useFavoritesStore());

  act(() => {
    result.current.addCity('Krakow');
    result.current.addCity('Krakow'); // drugi raz ten sam!
  });

  expect(result.current.favorites).toHaveLength(1); // tylko jeden
});

it('usuwa miasto z ulubionych', () => {
  useFavoritesStore.setState({ favorites: ['Warszawa', 'Krakow'] });
  const { result } = renderHook(() => useFavoritesStore());

  act(() => {
    result.current.removeCity('Warszawa');
  });

  expect(result.current.favorites).not.toContain('Warszawa');
  expect(result.current.favorites).toContain('Krakow');
});

# Uruchamianie testow:
npm test                                # Wszystkie testy (tryb watch)
npm test -- --watchAll=false            # Jednorazowe uruchomienie (CI)
npm test -- WeatherCard                 # Tylko pliki zawierajace 'WeatherCard'
npm test -- --coverage                   # Raport pokrycia kodu
npm test -- --verbose                     # Wyświetl nazwy kazdego testu
```

## 9. Architektura projektu - Feature-First



W RN Lab 1 używaliśmy prostej struktury z katalogami `components/`, `hooks/`, `services/`. Ta struktura, zwana Layer-First, jest czytelna dla małych projektów. Przy większej aplikacji zalecana jest architektura Feature-First: każda funkcjonalność ma własny, samowystarczalny katalog.

| Layer-First (Lab 1)                                                                                       | Feature-First (rekomendowany)                                                                          |
|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <code>components/</code> , <code>hooks/</code> , <code>services/</code> - grupowanie po typie technicznym | <code>features/weather/</code> , <code>features/favorites/</code> - grupowanie po funkcji biznesowej   |
| Zmiana jednej funkcji dotyka wielu katalogów jednocześnie                                                 | Zmiana jednej funkcji ograniczona do jednego katalogu feature/                                         |
| Latwo dodawać pliki na początku projektu                                                                  | Latwo skalować: nowy feature = nowy katalog, brak konfliktu                                            |
| Plik <code>weatherCard.tsx</code> 'sasiaduje' z <code>profileCard.tsx</code> mimo że to inne features     | Wszystko związane z pogodą jest w <code>features/weather/</code> - nawigacja po kodzie jest intuicyjna |

## 9.1. Struktura Feature-First dla WeatherApp v2

```

WeatherApp/
├── app/                                ← Expo Router: tylko routing (najcieniejsza warstwa)
│   ├── _layout.tsx                    ← QueryClient + GestureHandler + Providers
│   └── (tabs)/
│       ├── _layout.tsx                ← Konfiguracja zakładek
│       ├── index.tsx                  ← import { HomeScreen } from '@/features/home'
│       └── favorites.tsx              ← import { FavoritesScreen } from
│
│   '@/features/favorites'
│   └── detail/[city].tsx              ← import { DetailScreen } from '@/features/weather'
│
├── features/
│   ├── weather/                       ← Feature: pogoda i prognoza
│   │   ├── api/
│   │   │   └── weatherApi.ts          ← Tylko wywołania HTTP (fetchWeather, fetchForecast)
│   │   ├── hooks/
│   │   │   ├── useWeatherQuery.ts    ← TanStack Query: aktualna pogoda
│   │   │   └── useForecastQuery.ts   ← TanStack Query: prognoza 7 dni
│   │   ├── components/
│   │   │   ├── WeatherCard.tsx
│   │   │   └── ForecastTile.tsx
│   │   ├── screens/
│   │   │   └── DetailScreen.tsx
│   │   └── index.ts                   ← Barrel export: export { DetailScreen, WeatherCard
│   ├── favorites/                     ← Feature: ulubione miasta
│   │   ├── store/
│   │   │   └── favoritesStore.ts      ← Zustand store z persist
│   │   ├── components/
│   │   │   └── CityTile.tsx           ← Wiersz listy ulubionych
│   │   ├── screens/
│   │   │   ├── HomeScreen.tsx        ← Wyszukiwarka + lista ulubionych
│   │   │   └── FavoritesScreen.tsx
│   │   └── index.ts
│   └── settings/                      ← Feature: ustawienia aplikacji
│       ├── store/
│       │   ├── settingsStore.ts      ← Zustand: theme, language, units
│       └── screens/
│           └── SettingsScreen.tsx

```



## 10. Dostępność i internacjonalizacja

### 10.1. Dostępność - accessibilityLabel i rola

React Native generuje drzewo dostępności dla TalkBack (Android) i VoiceOver (iOS). Dla standardowych komponentów (Text, TextInput, Button) drzewo tworzone jest automatycznie. Dla niestandardowych elementów klikalnych i obrazów musisz opisać je ręcznie.

```

// Karta pogody z pełną dostępnością:
<Pressable
  onPress={onPress}
  accessible={true} // Element widoczny dla czytnika (domyślne true)
  accessibilityRole='button' // Czytnik ogłasza 'przycisk' i można aktywować
  accessibilityLabel={`Pogoda w ${weather.city}: ${weather.temperature} stopni, wilgotność ${weather.humidity} procent`}
  // Label czyta cały opis zamiast poszczególnych dzieci
  accessibilityHint='Aktywuj aby zobaczyć szczegóły prognozy'
  // Hint = co się stanie po aktywacji
>
  {/* Elementy dekoracyjne - ukryj przed czytnikiem */}
  <Image
    source={weatherIcon}
    accessibilityElementsHidden={true} // Android: ukryj cały element
    importantForAccessibility='no' // iOS: ignoruj ten element
    style={styles.icon}
  />
  {/* Tekst - czytnik przeczyta automatycznie gdy nie ma label na rodzicu */}
  <Text style={styles.city}>{weather.city}</Text>
  <Text style={styles.temp}>{weather.temperature}°C</Text>
</Pressable>
  
```

### 10.2. Internacjonalizacja z i18next i expo-localization

```

# Instalacja:
npm install i18next react-i18next
npm install expo-localization # Odczyt języka systemu

// i18n/config.ts - konfiguracja
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
  
```

```
import * as Localization from 'expo-localization';

const resources = {
  pl: { translation: {
    appTitle: 'Aplikacja Pogodowa',
    searchPlaceholder: 'Wpisz nazwe miasta...',
    humidity: 'Wilgotnosc: {{value}}%', // {{value}} = interpolacja
    lastUpdated: 'Zaktualizowano: {{date, datetime}}', // formatowanie daty
  }},
  en: { translation: {
    appTitle: 'Weather App',
    searchPlaceholder: 'Enter city name...',
    humidity: 'Humidity: {{value}}%',
    lastUpdated: 'Updated: {{date, datetime}}',
  }},
};

i18n.use(initReactI18next).init({
  resources,
  lng: Localization.getLocales()[0].languageCode ?? 'pl',
  fallbackLng: 'pl',
  interpolation: { escapeValue: false },
});

// Uzycie w komponencie:
import { useTranslation } from 'react-i18next';

const WeatherCard = ({ weather }: WeatherCardProps) => {
  const { t } = useTranslation();

  return (
    <View>
      <Text>{t('humidity', { value: weather.humidity })}</Text>
      <Text>{t('lastUpdated', { date: weather.updatedAt })}</Text>
    </View>
  );
};
```

## 11. React Native DevTools - diagnostyka i profilowanie

| Narzedzie               | Jak uzyc                                                          | Do czego sluzy                                                                     |
|-------------------------|-------------------------------------------------------------------|------------------------------------------------------------------------------------|
| React Native DevTools   | Shake emulatora → Open DevTools lub j w terminalu Expo            | Debugowanie JS: breakpointy, console.log, profiler wydajnosci, inspekcja sieci.    |
| React Developer Tools   | npm install -g react-devtools, potem react-devtools w terminalu   | Drzewo komponentow na zywo. Inspekcja props i stanu. Zmiana props na zywo.         |
| Flipper                 | Pobierz z fbflipper.com, uruchom i polacz z emulatorem            | Inspekcja sieci (wszystkie HTTP), baza danych, crash logi, pluginy spolecznosci.   |
| Zustand DevTools        | Dodatek Chrome/Firefox 'Redux DevTools' + simple-zustand-devtools | Podglad i time-travel historii stanu Zustand store.                                |
| TanStack Query DevTools | npm install @tanstack/react-query-devtools (web only)             | Stan cache: kto jest swiezy, kto stale, kiedy ostatni fetch.                       |
| Expo Dev Menu           | Shake emulatora lub Ctrl+M (Android) / Cmd+D (iOS sim)            | Fast Refresh, Perf Monitor, Element Inspector (zaznacz widget i zobacz jego styl). |

### 11.1. Profilowanie wydajnosci

```
# Performance Monitor (Expo Dev Menu > Perf Monitor):
# RAM: zuzycie pamieci w MB
# JS: czas wykonania watku JS w ms na klatke (cel: < 16ms dla 60fps)
# UI: czas renderowania watku UI (cel: < 16ms)
# FPS: aktualna czestotliwosc klatek

# Why-did-you-render - wykrywanie zbednych renderowan:
npm install --save-dev @welldone-software/why-did-you-render

// App.tsx lub index.ts (przed importem React):
if (__DEV__) {
  const whyDidYouRender = require('@welldone-software/why-did-you-render');
  whyDidYouRender(React, {
    trackAllPureComponents: true,
    logOnDifferentValues: true, // Loguj gdy props rozni sie referencja ale nie wartoscia
  });
}

// Na wybranym komponencie:
WeatherCard.whyDidYouRender = true;
// Konsola pokaże: 'WeatherCard re-rendered because of props change: weather (same value)'
```

## 12. Zadania do wykonania

Zadania sa kontynuacja projektu WeatherApp z React Native Lab 1. Kod z Lab 1 jest punktem startowym. Migrujemy go do nowej architektury i rozbudowujemy o nowe funkcje.

## Zadanie 1 (20 pkt) - Migracja na Zustand

1. Dodaj pakiety: `zustand`, `react-native-mmkv` (lub `@react-native-async-storage/async-storage` jeśli używasz Expo Go), `@tanstack/react-query`.
2. Utwórz `stores/favoritesStore.ts` z Zustand + persist. Pole `favorites: string[]`. Akcje: `addCity()`, `removeCity()`, `clear()`. Persystencja przez `AsyncStorage` lub `MMKV`.
3. Usun `FavoritesContext.tsx` i `FavoritesProvider`. Zastąp wszystkie użycia `useContext(FavoritesContext)` wywołaniami `useFavoritesStore(sel)`.
4. Skonfiguruj `QueryClient` w `app/_layout.tsx`. Utwórz `hooks/useWeatherQuery.ts` z `useQuery`. Usun ręczne `useEffect` + `useState` do pobierania pogody tam gdzie to możliwe.
5. Przetestuj że ulubione są zachowywane po zamknięciu i reopenowaniu aplikacji.

WERYFIKACJA: Brak importów z `FavoritesContext`. Ulubione przeżyją restart aplikacji. `useWeatherQuery` zwraca `isLoading`, `isError`, data bez ręcznie pisanego stanu.

## Zadanie 2 (25 pkt) - Offline i zaawansowany store

6. Rozszerz `weatherStore.ts` o mapę cache: `weatherMap: Record<string, WeatherData & { cachedAt: Date }>`. Zaimplementuj logikę 'cache-first': pokazuj stare dane natychmiast, pobieraj świeże w tle jeśli starsze niż 15 minut.
7. Dodaj wskaźnik świeżości danych (badge) w `WeatherCard`: 'Dane sprzed X minut'. Oblicz różnicę między aktualnym czasem a `cachedAt`.
8. Zaimplementuj Zustand store dla ustawień (`settingsStore.ts`): `units ('celsius' | 'fahrenheit')`, `language ('pl' | 'en')`. Przeliczaj temperaturę w zależności od `units`.
9. Przetestuj offline-first: załaduj pogodę, wyłącz internet (tryb samolotowy), uruchom ponownie - stare dane powinny się wyświetlić z odpowiednim komunikatem.

WERYFIKACJA: Dane widoczne przy wyłączonym internecie. Badge z wiekiem danych. Przełącznik Celsjusz/Fahrenheit działa.

## Zadanie 3 (30 pkt) - Animacje

10. Animowane wejście kart na `HomeScreen`: każda karta wchodzi z przesunięcia 80px z prawej z opóźnieniem `index * 70ms`. Użyj `Animated.parallel (translateX + opacity)`. Czas: 350ms, `Easing.out(Easing.back(1.5))`.
11. `WeatherCard` z efektem naciśnięcia: `scale 1.0 → 0.95` przy `onPressIn` (`Animated.spring`, `bounciness: 0`), powrót `0.95 → 1.0` przy `onPressOut` (`bounciness: 4`).
12. Zainstaluj `react-native-reanimated` i `react-native-gesture-handler`. Zaimplementuj `SwipeableCard` na ekranie Ulubionych: przeciągnięcie  $\geq 100px$  w lewo usuwa miasto z listy (`Reanimated + Gesture.Pan()`).
13. Opcjonalnie: animacja rozwijania szczegółów w `WeatherCard` - po tapnięciu karta rozszerza się pionowo ujawniając wilgotność i wiatr (`Animated.timing` na `height` lub `withTiming` na `SharedValue`).

WERYFIKACJA: Demo na AVD przy 60fps. Staggered entry lista. Scale przy naciśnięciu. Swipe-to-delete na Ulubionych. Reanimated skonfigurowany (`babel.config.js`).

## Zadanie 4 (25 pkt) - Testy

14. Napisz min. 4 testy jednostkowe dla favoritesStore.ts: addCity (normalny), addCity (duplikat), removeCity, clear(). Użyj renderHook i setState do resetowania stanu między testami.
15. Napisz min. 3 testy komponentu WeatherCard: wyświetlanie miasta, temperatury, wywołanie onPress. Użyj mockWeather fixture.
16. Napisz test integracyjny HomeScreen: wpisanie miasta w SearchBar → submit → załadowanie danych (mock fetchWeather) → wyświetlenie WeatherCard. Użyj jest.mock('../services/weatherService') i waitFor.
17. Uruchom npm test --coverage. Osiągnij pokrycie > 60% dla pliku favoritesStore.ts i WeatherCard.tsx. Dołącz screenshot raportu do sprawozdania.

**WERYFIKACJA:** npm test uruchamia się bez błędów. Min. 8 testów (4 jednostkowe + 3 komponentów + 1 integracyjny). Raport coverage > 60%.

## 13. Kryteria oceniania

| Zadanie          | Punkty  | Co weryfikuje prowadzący                                                                                        |
|------------------|---------|-----------------------------------------------------------------------------------------------------------------|
| Zad. 1: Zustand  | 20 pkt  | Brak Context API. persist działa. QueryClient skonfigurowany. useWeatherQuery użyty zamiast useEffect+useState. |
| Zad. 2: Offline  | 25 pkt  | Dane widoczne przy wyłączonym internecie. Badge czasu. settingsStore z przelicznikiem jednostek.                |
| Zad. 3: Animacje | 30 pkt  | Staggered entry lista. Scale przy naciśnięciu. Swipe-to-delete z Reanimated. Babel plugin skonfigurowany.       |
| Zad. 4: Testy    | 25 pkt  | Min. 8 testów. Poprawne mockowanie. waitFor dla async. Coverage > 60%.                                          |
| RAZEM            | 100 pkt |                                                                                                                 |

| Ocena | Punkty / Wymagania                                                                                                 |
|-------|--------------------------------------------------------------------------------------------------------------------|
| 5.0   | 90-100 pkt - Wszystkie zadania. Reanimated + gesture. Coverage > 70%. i18next lub dostępność (accessibilityLabel). |
| 4.5   | 80-89 pkt - Zadania 1-3 kompletne. Min. 6 testów. Swipe-to-delete działa.                                          |
| 4.0   | 70-79 pkt - Zadania 1-2 + staggered entry (Animated API). Min. 4 testy.                                            |
| 3.5   | 60-69 pkt - Zadania 1-2. Zustand + persist działa. Brak animacji.                                                  |
| 3.0   | 50-59 pkt - Zadanie 1: Zustand zamiast Context. Ulubione persystują.                                               |
| 2.0   | 0-49 pkt - Projekt nie kompiluje lub Zustand nie działa.                                                           |

## 14. Najczęstsze błędy i ich rozwiązania

| Błąd / Objaw                                                               | Przyczyna i rozwiązanie                                                                                                                                                                                                                                                                         |
|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Zustand: Store rerenderuje cały komponent mimo selektora                   | Destrukturyzujesz zamiast używać selektora: <code>const { favorites } = useStore()</code> = anti-pattern! Użyj: <code>const favorites = useStore(s =&gt; s.favorites)</code> . Jeśli potrzebujesz wielu pól, użyj <code>useShallow</code> .                                                     |
| Reanimated: 'Tried to synchronously call a non-worklet function'           | Wywołales funkcje JS ( <code>setState</code> , nawigacja) bezpośrednio w worklet. Owin w <code>runOnJS()</code> : zamiast <code>navigation.push()</code> napisz <code>runOnJS(navigation.push)('/detail')</code> .                                                                              |
| Reanimated: Animacja nie działa po konfiguracji Babel                      | 'react-native-reanimated/plugin' nie jest ostatnim pluginem w <code>babel.config.js</code> . Upewnij się że jest na końcu listy plugins. Wyczyść cache: <code>npx expo start --clear</code> .                                                                                                   |
| TanStack Query: stale data po zmianie parametru                            | <code>queryKey</code> nie zawiera wszystkich parametrów wpływających na wynik. Zmien: <code>queryKey: ['weather']</code> na <code>queryKey: ['weather', city]</code> . Zmiana <code>city</code> automatycznie zainwaliduje cache.                                                               |
| MMKV: 'MMKV is not available in this environment'                          | MMKV wymaga native build i nie działa w Expo Go. Użyj <code>npx expo prebuild</code> i uruchom bezpośrednio przez <code>npx expo run:android</code> . Lub zamień na <code>AsyncStorage</code> dla testów w Expo Go.                                                                             |
| Jest: 'Cannot find module react-native-reanimated'                         | Reanimated wymaga konfiguracji w <code>jest transformIgnorePatterns</code> . Dodaj 'react-native-reanimated' do listy: <code>transformIgnorePatterns: ['node_modules/(?!(... react-native-reanimated)/)']</code> .                                                                              |
| RNLT: 'act() warning: Wrap in act(...)'                                    | Zmiana stanu poza <code>act()</code> . Owin każdą akcję zmieniającą stan w <code>act()</code> : <code>act(() =&gt; { result.current.addCity('Gdansk'); })</code> . Dla <code>async</code> : <code>await act(async () =&gt; { await loadData(); })</code> .                                      |
| Zustand persist: Stare dane po zmianie struktury store                     | Zmieniles kształt stanu ale stary JSON w <code>AsyncStorage</code> nadal istnieje. Dodaj wersjonowanie: <code>version: 1</code> w opcjach <code>persist</code> i funkcję <code>migrate()</code> , lub zmień <code>name</code> klucza.                                                           |
| <code>useAnimatedStyle</code> : 'Reading from value during render'         | Używasz <code>SharedValue.value</code> wewnątrz body komponentu (poza <code>useAnimatedStyle</code> ). <code>SharedValues</code> można czytać TYLKO w workletach ( <code>useAnimatedStyle</code> , <code>useDerivedValue</code> ) lub przez <code>.value</code> w <code>handleEvent</code> ach. |
| <code>GestureDetector</code> : Gest nie reaguje na <code>ScrollView</code> | Konflikty gestów: <code>ScrollView</code> i <code>Pan</code> gest walczą o to samo dotknięcie. Użyj <code>activeOffsetX</code> w <code>Pan</code> gesture aby rozróżnić od pionowego scrolla. Lub owin <code>ScrollView</code> w <code>NativeViewGestureHandler</code> .                        |

### Narzędzia diagnostyczne

```
# Weryfikacja konfiguracji Babel (czy Reanimated jest ostatni):
cat babel.config.js

# Reset pełny (gdzieś coś dziwnego dzieje się z pakietami):
rm -rf node_modules .expo
npm install
npx expo start --clear

# Sprawdź czy MMKV jest dostępne (wymaga native build):
npx expo run:android # Zamiast npx expo start

# Debugowanie Zustand persist - sprawdź co jest w AsyncStorage:
import AsyncStorage from '@react-native-async-storage/async-storage';
AsyncStorage.getAllKeys().then(console.log);
```

```
AsyncStorage.getItem('favorites-storage').then(console.log);  
  
# Profil rerenderow (why-did-you-render musi byc skonfigurowany):  
WeatherCard.whyDidYouRender = true;  
  
# Testy z verbose output:  
npm test -- --verbose --watchAll=false
```

---

## Instrukcja Laboratoryjna React Native 2 - Zustand, Reanimated, AsyncStorage i Testy

Programowanie Aplikacji Mobilnych | Katedra Informatyki i Automatyki

**Nastepne cwiczenie: React Native 3 - Publikacja w App Store i Play Store, EAS Build, CI/CD**